AURA BI: AI Business Intelligence & Research Consultant Platform

Comprehensive Technical & Architectural Documentation

Prepared for: **Unada Labs Pvt Ltd**

Prepared by: AURA BI Core Engineering Team

Date: July 2026

CONFIDENTIALITY NOTICE: The information contained in this document is proprietary to the engineering group and intended for assessment and integration evaluations at Unada Labs Pvt Ltd. Unauthorized copying, distribution or usage is strictly prohibited.

1. Executive Summary & Approach

AURA BI is a production-quality, multi-agent AI system designed to automate corporate research, strategic analysis, and executive pitch creation. By accepting a company name as input, the system crawls the web, extracts structured financial and strategic profiles, conducts SWOT and competitor analyses, recommends AI-driven opportunities with estimated ROI, and drafts a personalized pitch memo addressed to the CEO.

Our approach centers on three architectural principles:

- **Agentic Specialization (Swarm Architecture):** Rather than utilizing a single monolithic LLM prompt, we decompose corporate intelligence gathering into **six specialized, sequential agents**. Each agent operates with specific scopes, context boundaries, and distinct search tasks.
- **Deterministic State Synchronization:** We employ a structured StateGraph pattern. The state accumulates sequentially in a typed context dictionary, which prevents context drift and ensures stable data schemas.
- **Hybrid Execution Resilience:** To guarantee instant usability, the platform supports both **Live Agent Mode** (using active search and live LLM keys) and a **Dynamic Demo Mode** (which generates high-fidelity simulated models with zero API dependencies).

2. System Architecture & Workflow

The platform is built on a decoupled **Client-Server Architecture**:

- **Frontend Client:** A React 19 single-page application configured with TypeScript and Vite. It utilizes Tailwind CSS for its glassmorphism design system, Framer Motion for step-by-step agent loading animations, and Recharts for responsive SVG financial charts.
- **Backend Server:** A FastAPI server running on Uvicorn. It exposes endpoints to trigger search routines, query search history, and export reports directly into styled PDF and PowerPoint binary streams in-memory.

Agentic Graph Execution Steps

1. **Research Agent:** Gathers company facts, industry sectors, headquarters locations, products, revenue indexes, and leadership structures via Tavily search.
2. **News Agent:** Discovers the top 3 recent product launches, mergers, acquisitions, and expansions.
3. **Business Analyst Agent:** Evaluates internal strengths, operational bottlenecks, and external threats to formulate a SWOT matrix.
4. **Competitor Agent:** Identifies and ranks top sector competitors, sizing up their employee scale, tech stacks, and market positions.
5. **AI Consultant Agent:** Analyzes the company's weaknesses and proposes exactly 3 concrete, ROI-driven AI solutions.
6. **CEO Pitch Agent:** Formulates a personalized executive summary and a consulting memo addressed directly to the C-suite.

3. AI & Developer Tools Used

The technology stack is carefully selected to support real-time data flow, high-performance structured processing, and polished visualization.

Category	Tool / Library	Purpose in AURA BI
Orchestration	LangGraph	Coordinates the 6-agent sequential workflow, maintains thread state, and controls node execution.
LLM Integration	LangChain (Core)	Standardizes prompting, model loading, and API connections.
Search Engine	Tavily API	Serves as an optimized search agent that returns cleaned, scraped HTML text directly from web indexes.
AI Models	Google Gemini 1.5 Flash / OpenAI GPT-4o-mini	Powers the core reasoning, SWOT classification, competitive analysis, and text generation.
Backend API	FastAPI / Uvicorn	Serves high-performance REST endpoints with automated Swagger UI docs.
Data Schema	Pydantic (v2)	Enforces strict type casting and validation on nested agent outputs.
Document Export	ReportLab & python-pptx	Generates styled corporate PDF files and PowerPoint presentations in-memory as binary streams.
Frontend Client	React 19 / TypeScript / Vite	Modern browser interface ensuring type safety and sub-second builds.

4. Key Challenges Faced & Solutions

1. LLM Schema Hallucinations & Format Deviation

Large language models often wrap JSON responses in markdown backticks or include conversational preambles. This breaks programmatic JSON parsing.
Solution: We designed a *call_llm_json* helper function which appends strict prompt instructions, strips markdown tags using string sanitization, and implements safe fallbacks to prevent 500 server crashes.

2. Search Result Ambiguity (Company Name Pollution)

Searching for a company with multiple active branches or common names (e.g. 'Reliance') can pollute the search context with mixed and inaccurate data.
Solution: We optimized the Research Agent prompt with strict boundaries. It forces the LLM to identify the main parent entity, ignore mixed subsidiaries, and query specific terms sequentially (e.g. headquarters, revenue).

3. Cold-Start Experience (API Key Dependencies)

Without active API keys (Gemini, OpenAI, Tavily), the application would fail, ruining the first-use evaluation experience.
Solution: We built a *Dynamic Demo Engine*. If credentials are missing, the system transitions to Demo Mode, serving high-fidelity presets for popular companies (Stripe, Tesla, NVIDIA, Apple, Jio) and generating realistic, hash-based mock profiles for new companies.

4. In-Memory Document Export Compilation

Compiling PDFs and PPTX files directly on the server's disk slow down response times and create storage safety challenges in cloud deployments.
Solution: All exports are processed directly in-memory using *io.BytesIO* streams. FastAPI streams these bytes directly to the client browser, resulting in zero disk usage and instant downloads.

5. Execution Walkthrough

The platform operates through a simple, high-fidelity lifecycle:

1. **User Input & Loading:** The user enters a company name in the React web dashboard. The client triggers a loading screen tracking the individual progress of the multi-agent graph.
2. **FastAPI Router:** The request hits the */api/search* endpoint. If API keys are present, it runs the LangGraph engine; otherwise, it redirects to the Dynamic Demo Engine.
3. **Sequential Execution:** The Research, News, Analyst, Competitor, Consultant, and Pitch nodes execute sequentially. Their output is aggregated, validated against a Pydantic schema, and returned to the client as JSON.
4. **UI Presentation:** The frontend renders SWOT tables, competitor benchmarking cards, executive charts using Recharts, and the formal CEO pitch memo.
5. **On-Demand Exporting:** Clicking 'Export PDF' or 'Export PPT' serializes the active dashboard report, posts it to the backend, builds the styled document in-memory, and triggers a file download in the browser.